```
sensorValue = analogRead(A0);
```

Or we might use the function inside another. Say for instance, we wanted our pin 13 LED to blink slowly when it was dark and quickly when there was light. With a light sensor connected to pin A0, our code could look as follows:

```
delay(analogRead(A0));
```

Here we will delay for the amount of time as specified by the integer received from analog input pin A0 in an amount from 0 to a little over 1 second. To extrapolate this into a full code example, have a look at Listing 6-2.

Listing 6-2. *analogRead() Example*

```
int analogValue = 0;
int ledPin = 13;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  analogValue = analogRead(A0);
  digitalWrite(ledPin, HIGH);
  delay(analogValue);
  digitalWrite(ledPin, LOW);
  delay(analogValue);
}
```

This is the basic blink sketch, except now the LED will blink slowly when an analog sensor provides a low-voltage reading and blink quickly when the sensor provides a higher voltage reading. This works by adding an `analogRead()` statement that assigns an analog value to the variable `analogValue` that we use as a length of time in the `delay()` function. What analog sensor is used for in this example is up to you, but our wind sensor from our project would work great. So, now that we can read these values, let's look at how can we output analog values.

analogWrite()

The function `analogWrite()` will allow us to access the pulse width modulation hardware on the Arduino microcontroller. The basic syntax for this function follows:

```
analogWrite(pin, duty cycle)
```

In using this function we need to give it two pieces of information. The first is the pin number, which can only include one of the following pins on the Arduino Uno: 3, 5, 6, 9, 10, and 11. These are marked as PWM on the interface board. Other versions of the Arduino may have fewer or more PWM pins available, so double-check your hardware's documentation. The second bit of information is the duty cycle expressed as an 8-bit numerical integer ranging between the values of 0 and 255. The value 0 corresponds to off or 0% duty cycle and 255 is basically full on or 100% duty cycle. Any value in between these two endpoints provides a corresponding duty cycle approximating an analog output.